Motor Vehicle Insurance Claims

Classification Analysis

Applied Machine Learning | CU Boulder

1. Introduction

This project applies classification to a motor vehicle insurance dataset sourced from ICPSR to predict whether a customer will file an insurance claim. The dataset contains about 105,555 records with 30 features covering customer demographics, policy details, and vehicle characteristics. The dataset holds the primary activities over a span of three full years (November 2015 to December 2018) concerning non-life motor insurance portfolio. Each row signifies a policy transaction, while each column represents a distinct variable.

You can see the referenced variable definitions here (provided on the dataset's documentation page):

Descriptive of the variables.xlsx

The primary question I am trying to answer is: *given a customer profile, can we predict whether they are at risk of needing to file an (insurance) claim?*

2. Dataset

2.1 Data Source

Dataset: Motor Vehicle Insurance Data
Source: ICPSR
Records: 105,555 rows | 30 columns

	ID	Date_start_contract	Date_last_renewal	Date_next_renewal	Date_birth	Date_driving_licence	Distribution_channel	Seniority	Policies_in_force	Max_policies	...
0	1	05/11/2015	05/11/2015	05/11/2016	15/04/1956	20/03/1976	0	4	1	2	...
1	1	05/11/2015	05/11/2016	05/11/2017	15/04/1956	20/03/1976	0	4	1	2	...
2	1	05/11/2015	05/11/2017	05/11/2018	15/04/1956	20/03/1976	0	4	2	2	...
3	1	05/11/2015	05/11/2018	05/11/2019	15/04/1956	20/03/1976	0	4	2	2	...
4	2	26/09/2017	26/09/2017	26/09/2018	15/04/1956	20/03/1976	0	4	2	2	...

2.2 [Target] Variable

My target column for this set is a new column I made called "Target" which is a reduction of the N_claims_history column into binary categories. The N_claims_history variable indications the total amount of claims filed over the three year period so we should squash it down:

1 = Customer filed at least one claim in a three year period
0 = Customer did not file a claim

We can achieve this by the following:

```
df['[Target]'] = (df['N_claims_year'] > 0).astype(int)
```

3. Feature Engineering

Before modeling, I needed to take a few preprocessing steps to maximize model performance and follow general data science 'good habits'. Each decision is justified below.

3.1 High Correlation Variables

A quick check of correlation between variables led to the removal of only one column:

> Cylinder_Capacity

We want to remove highly correlated variables because they bog down the model and one can do the heavy lifting for both in this case. We were particularly interested in variables with a correlation coefficient of 0.75 or more to indicate 'high correlation'.

3.2 Converting Dates

Some of the raw date columns were converted into meaningful numeric features:

> Age: Derived from Date_birth relative to reference date 2024-01-01
> Driving_experience: Derived from Date_driving_licence, representing years of experience

The idea here being that date formats like xx/XX/xxxx are not useful to our model as is but we can turn them into numeric values that represent their value to the model:

```
# Engineer age features
df['Date_birth'] = pd.to_datetime(df['Date_birth'], errors='coerce')
df['Date_driving_licence'] = pd.to_datetime(df['Date_driving_licence'],
errors='coerce')
reference_date = pd.Timestamp('2024-01-01')
df['Age'] = (reference_date - df['Date_birth']).dt.days / 365
df['Driving_experience'] = (reference_date -
df['Date_driving_licence']).dt.days / 365
```

The rest of our date formats will be dropped to avoid errors later.

3.3 Feature Engineering

With a large and versatile dataset we find ourselves with lots of string typed data. The methods to deal with string based data while keeping variable relationship integrity is through both one-hot encoding and ordinal encoding. By carefully sifting through my string based data I made the decision to put the ['Area'] column under ordinal data because while it contains 0/1 data values already, 0 is meant for smaller vehicles and 1 for much larger:

```
ordinal_cols = ['Area']
oe = OrdinalEncoder()
df[ordinal_cols] = oe.fit_transform(df[ordinal_cols])
```

However, the remainder of my string based variables do not have a strong reason for being ordinal and fit the definition of categorial instead:

```
cat_cols = ['Distribution_channel','Type_fuel', 'Second_driver',
'Type_risk', 'Payment']
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
```

3.4 Feature Selection

I dropped several other noticeable columns that were either too similar or used to directly create our [‘Target’] column. The following were just cut from the dataset:

ID: Unique identifier with no use case.
Date_Lapse: Date format of when a lapse occurred, not useful.
N_claims_history: Source of the [‘Target’] column itself
N_claims_year / R_Claims_history: Too closely related to the [‘Target’] column but does not show up on the .corr radar because it is in date form.
Cost_claims_year: Too relevant in figuring out if a claim was made for claims that resulted in a cost.
Date columns already used by engineered features (Age, Driving_experience)
Remainder of the date columns that were not relevant.

```
drop_cols = ['ID', 'Date_start_contract', 'Date_last_renewal',
'Date_next_renewal', 'Date_lapse',
            'Date_birth', 'Date_driving_licence',
            'N_claims_year', 'N_claims_history', 'R_Claims_history']

df = df.drop(columns=drop_cols)
```

3.5 Missing Values

Approximately 10,000 rows contained missing values. Rather than dropping these rows and losing about 10% of the dataset, we can use KNN to fill in these values. This method fills missing values by averaging the values of the 3 most similar rows, preserving relationships between features better than simple mean imputation. Here's how we take care of it in my dataset:

```
#Deal with missing values, use KNN
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=3)
df_KNN= pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
df = df_KNN
```

4. Model Development

4.1 Train/Test Split

The dataset was split 80/20 into training and test sets using random_state=42 for consistency. The importance of a good train test split is to prevent overfitting with our model. Overfitting is when our model is too adjusted to fit the dataset it was given, but only for that dataset. The 80/20 split lets us see how the model does with data we still have, minus the overfitting:

```
#create X and Y dataframes
X = df.drop('Target', axis=1)
y = df['Target']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

4.2 Decision Tree Classifier / Balancing

I picked a Decision Tree Classifier for its easy interpretability and ability to capture non-linear relationships in the data. I also knew ahead of time I wanted to solve a 0/1 classification problem for identifying insurance claims so the DTC was a smooth decision that could further be visualized by a confusion matrix.

Another problem I ran into was the base balancing of the dataset. Our y_train.value.counts() would output this:

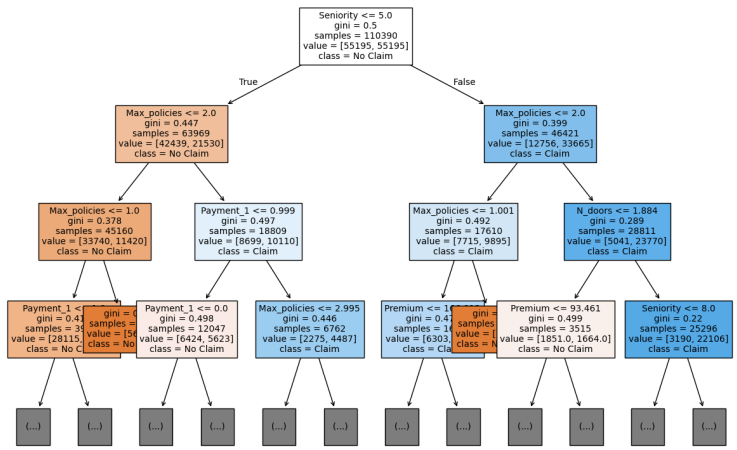
```
Target
1.0    55196
0.0    29248
Name: count, dtype: int64
```

Our dataset is imbalanced, dominated by the majority class, which could cause the model to predict it most of the time. To address this, we applied SMOTE, which uses K-Nearest Neighbors to generate synthetic minority-class samples. While we were not taught a specific balancing method in class, maintaining a balanced dataset is crucial for accurate predictions:

```
print(y_train.value_counts())

Target
1.0    55196
0.0    29248
Name: count, dtype: int64

smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

print(y_train.value_counts())

Target
1.0    55196
0.0    55196
Name: count, dtype: int64
```

4.3 Decision Tree Visualization

The decision tree was visualized at max_depth=3 for readability. This shows the top-level decision rules the model learned without rendering the full tree which would be unreadable.



This tree keeps its simple for identifying the highest impact variables on insurance claims with Seniority being at the top. A quick assessment of this allows us to identify the Seniority variable as the number of years a customer has been with their insurance company. We can reasonably assume this means someone who keeps the same insurance company is happy with their arrangement and has not needed to pay large fees and fines. *Or claims.*

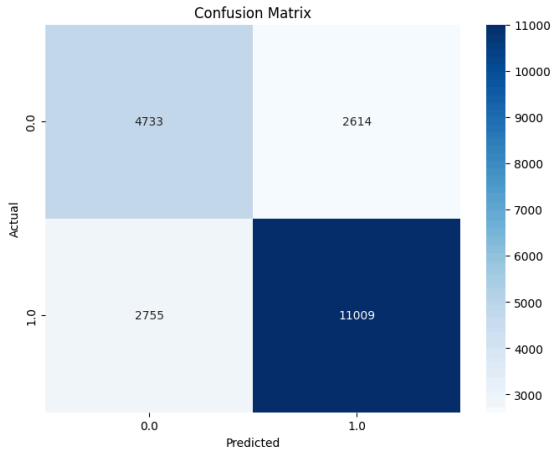
5. Model Evaluation

5.1 Classification Report

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label=1.0)
recall = recall_score(y_test, y_pred, pos_label=1.0)
f1 = f1_score(y_test, y_pred, pos_label=1.0)

print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1:.4f}")

Accuracy:  0.7457
Precision: 0.8081
```

```
Recall:    0.7998
F1-Score:  0.8040
```

5.2 Confusion Matrix



5.3 Results Discussion

The model performed well, achieving 74.6% accuracy, 80.8% precision, and 79.9% recall. Precision indicates that when the model predicts a claim, it is correct 8 out of 10 times, while recall shows it identifies roughly 8 out of 10 actual claims. The F1 score reflects a balanced performance between these metrics. The confusion matrix confirms this, with most predictions falling along the diagonal, indicating the model is correctly classifying the majority of test cases.

6. Conclusion

This project applied a Decision Tree classifier to predict whether a motor vehicle insurance customer would file a claim. After preprocessing, encoding categorical variables, engineering date features, and addressing class imbalance with SMOTE, the model achieved strong performance on the test set, with 74.6% accuracy, 80.8% precision, and 79.9% recall and key features such as Seniority were identified as important predictors. Overall, this analysis shows that machine learning can effectively be tasked with decision-making in insurance risk assessment if needed.